

Subject Description Form

Subject Code	COMP1002			
Subject Title	Computational Thinking and Problem Solving			
Credit Value	4			
Level	1			
Pre-requisite / Co-requisite / Exclusion	Nil			
Objectives	The objectives of this subject are to: - equip students with no prior experience on computer programming with fundamental computational and skills. In particular, the students will learn how to abstract and solve problems, and how to implement them in a high-level programming language; and - enable students to be competent lifelong problem solvers.			
Intended Learning Outcomes	Upon completion of the subject, students will be able to: <u>Professional/academic knowledge and skills:</u> - show a basic understanding of concepts of computational thinking; - understand the nature and characteristics of real-life problems and model them as computational problems; - develop computer solutions to basic and standard problems and implement them using a high-level programming language, e.g. Python; - acquire some specialised programming skills to implement solutions using suitable data types and constructs; and - recognise the properties of a competent problem solver and self-learner.			
Subject Synopsis/ Indicative Syllabus	<table><tr><td>Topic</td></tr><tr><td>1. Introduction to Computational Thinking Formulating problems for computers to solve them; logically organizing and analysing data; representing data through abstractions; automating solutions through algorithmic thinking; implementing efficient solutions; generalizing the problem-solving process.</td></tr><tr><td>2. Problem Solving through a High-level Programming Language Computing with numbers and strings; lists and files; functions; decision structures; loop structures and Booleans; sets and dictionaries.</td></tr></table>	Topic	1. Introduction to Computational Thinking Formulating problems for computers to solve them; logically organizing and analysing data; representing data through abstractions; automating solutions through algorithmic thinking; implementing efficient solutions; generalizing the problem-solving process.	2. Problem Solving through a High-level Programming Language Computing with numbers and strings; lists and files; functions; decision structures; loop structures and Booleans; sets and dictionaries.
Topic				
1. Introduction to Computational Thinking Formulating problems for computers to solve them; logically organizing and analysing data; representing data through abstractions; automating solutions through algorithmic thinking; implementing efficient solutions; generalizing the problem-solving process.				
2. Problem Solving through a High-level Programming Language Computing with numbers and strings; lists and files; functions; decision structures; loop structures and Booleans; sets and dictionaries.				

	3. Program Design Problem analysis and design; function abstraction and modularisation; bottom up and top down approaches.						
	4. Application of Computational Techniques Applications in different domains, for example, financial data computing, puzzle solving, development of games, web development, and scientific computation.						
Teaching/ Learning Methodology	The lectures will cover the main concepts and ideas in solving problems with computers and illustrate them using many examples. The students will also be given time to practice those concepts and ideas right away. The laboratory will be used to mainly cover program design and implementation of the theoretical concepts.						
Assessment Methods in Alignment with Intended Learning Outcomes	Specific assessment methods/tasks	% weighting	Intended subject learning outcomes to be assessed				
			a	b	c	d	e
	Continuous Assessment [such as quizzes, assignments etc.]	30%	✓	✓	✓	✓	✓
	Examination	70%	✓		✓	✓	
	Total	100%					
	Explanation of the appropriateness of the assessment methods in assessing the intended learning outcomes: Continuous Assessment (including pre-class, in-class, labs, and quizzes etc.) is designed to achieve learning outcomes (a), (b), (c), (d), and (e) through active engagement and practical application. The examination will cover (a), (c), and (d), assessing foundational understanding, solution development, and programming skills.						
	Student Study Effort Expected						
	Class contact:						
	▪ Lecture/Lab/Tutorials				39 Hrs.		
	Other student study effort:						
▪ Assignments, Tests, Examination				53 Hrs.			
▪ Self-Study				48 Hrs.			
Total student study effort				140 Hrs.			
Reading List and References	Reference Books: 1. Kowalski, Robert, <i>Computational Logic and Human Thinking: How to be Artificially Intelligent</i> , 1 st Edition, Cambridge University Press, 2011.						

	2. Dromey, R. G., <i>How to Solve It by Computer</i>. Prentice-Hall International, Englewood Cliffs, NJ, USA, 1982. (There is a free copy online.) 3. Zelle, John, <i>Python Programming: An Introduction to Computer Science</i> 3rd Edition. Franklin, Beedle & Associates Inc., 2017. 4. Downey, Allen B., <i>Think Python: How to Think Like a Computer Scientist</i>, Green Tea Press, 2015. 5. Punch, William F. and Enbody, Richard, <i>The Practice of Computing Using Python</i>, 3rd Edition, Addison Wesley, 2017. 6. Gries, Paul, Campbell, Jennifer and Montojo, Jason, <i>Practical Programming: An Introduction to Computer Science Using Python 3.6</i>. Pragmatic Bookshelf, 3rd Edition, 2017.
--	---